

Documentation of High-Fidelity Upgrades to the pybuildingenergy Pipeline

Sarthak Shrivastava

July 9, 2026

Abstract

This document outlines the modifications made to the baseline ISO 52016-1 simplified hourly calculation method within the `pybuildingenergy` codebase. The primary objective of these changes was to resolve the standard’s tendency to systematically overestimate peak cooling loads and underestimate passive heat dissipation. By injecting high-fidelity dynamic models—such as time-variant window properties, non-linear sky radiation, and variable air density—the pipeline now aligns closely with detailed dynamic simulation engines like EnergyPlus.

Contents

1	Dynamic Window Properties	3
1.1	Reasoning	3
1.2	Code Implementation	3
2	Escaping Solar Radiation	3
2.1	Reasoning	3
2.2	Code Implementation	3
3	Split Solar Transmission and Glass Absorption	4
3.1	Reasoning	4
3.2	Code Implementation	4
4	Variable Air Density for Ventilation	4
4.1	Reasoning	4
4.2	Code Implementation	4
5	Non-Linear Long-Wave Sky Radiation	4
5.1	Reasoning	4
5.2	Code Implementation	5
6	Transient Heat Accumulation (Thermal Mass Skew)	5
6.1	Reasoning	5
6.2	Code Implementation	5

7	Coupling of Thermally Conditioned Zones	5
7.1	Reasoning	5
7.2	Code Implementation	6
8	Subsequent changes in the heating/cooling load	6

1 Dynamic Window Properties

1.1 Reasoning

The baseline ISO 52016 standard assumes constant values for the thermal transmittance (U_{win}) and the total solar energy transmittance (g_{win}). In reality, these properties fluctuate based on wind speed and the solar angle of incidence. Using a solar-angle dependent correction factor (F_W) prevents the model from over-admitting solar heat at steep sun angles.

1.2 Code Implementation

The dynamic properties are calculated using the Karlsson and Roos empirical model inside `pybui/functions.py` and injected into the hourly loop in `pybui/utils.py`:

```
1 # Mapping orientation to azimuth and calculating dynamic properties
2 az_map = {"NV": 0.0, "EV": 90.0, "SV": 180.0, "WV": 270.0, "HOR": 0.0}
3 current_window_azimuth = az_map.get(orientation_elements[Eli], 0.0)
4
5 g_dyn, u_dyn = dynamic_window_properties(
6     g_normal=g_gl_wi_t[Eli],
7     u_nominal=building_object["building_surface"][Eli]["u_value"],
8     alpha_sol_t=sim_df['solar_altitude'].iloc[Tstepi],
9     phi_sol_t=sim_df['solar_azimuth'].iloc[Tstepi],
10    beta_k_t=float(building_object["building_surface"][Eli]["
orientation"] ["tilt"]),
11    gamma_k_t=current_window_azimuth,
12    wind_speed_m_s=sim_df['WS10m'].iloc[Tstepi]
13 )
```

2 Escaping Solar Radiation

2.1 Reasoning

The standard assumes that 100% of solar radiation transmitted through windows is trapped inside the thermal zone. In high-fidelity modeling, a portion of this shortwave radiation reflects off internal floors and walls and escapes back out through the glazing. Neglecting this artificially inflates the summer cooling load.

2.2 Code Implementation

An internal reflectance factor of 15% (0.15) was introduced to the internal solar gain accumulator.

```
1 # Introduce an internal reflectance factor to prevent a 100% heat trap
2 internal_reflectance = 0.15
3
4 Phi_sol_dir_zt_t += tau_win * (
5     sim_df[f'I_sol_dif_{orientation_elements[Eli]}'].iloc[Tstepi] +
6     sim_df[f'I_sol_dir_w_{orientation_elements[Eli]}'].iloc[Tstepi] *
7     F_sh_obst_wi_t
8 ) * area_elements[Eli] * (1 - Ffr_wi) * (1.0 - internal_reflectance)
```

3 Split Solar Transmission and Glass Absorption

3.1 Reasoning

The standard applies the total g_{win} directly to the interior air and surface nodes instantly. Physically, a window directly transmits a portion of the solar energy as light, but absorbs the rest into the glass pane. The absorbed heat conducts into the room gradually, creating thermal lag.

3.2 Code Implementation

The dynamic g-value was split into a transmission coefficient (τ) and an absorption coefficient (α). The absorbed portion was routed to the exterior thermal node of the glass to be subjected to convective wind cooling before entering the zone.

```
1 # Split the dynamic g-value based on standard double-glazing physics
2 tau_win = 0.85 * g_dyn # 85% is directly transmitted light
3 alpha_win = 0.15 * g_dyn # 15% is absorbed heat in the glass pane
4
5 # Override the exterior node's absorption coefficient
6 a_sol_pli_eli[0, Eli] = alpha_win * (1 - Ffr_wi)
```

4 Variable Air Density for Ventilation

4.1 Reasoning

Ventilation heat transfer calculations in the ISO standard use a constant air density (1.204 kg/m³). Because hot summer air is less dense, this assumption overestimates the thermal mass of incoming outdoor air during peak cooling periods.

4.2 Code Implementation

The Ideal Gas Law was implemented in `pybui/ventilation.py` to calculate the exact density of the air mixture dynamically based on the hourly zone and outdoor temperatures.

```
1 # Dynamic calculation using the Ideal Gas Law
2 T_avg_K = ((float(Tz) + float(Te)) / 2.0) + 273.15
3 dynamic_rho_air = 101325.0 / (287.05 * T_avg_K)
4
5 # Applied to the heat transfer coefficient [W/K]
6 Hve_k_t = c_air * dynamic_rho_air * (qv_arg_in_m3_h / 3600.0)
```

5 Non-Linear Long-Wave Sky Radiation

5.1 Reasoning

The baseline model assumes a constant 11 K offset between the outdoor air and apparent sky temperature. This severely underestimates radiative heat dumping at night. Replac-

ing this with a non-linear formulation (Swinbank model) accurately captures night-time radiant cooling.

5.2 Code Implementation

The calculation for ‘ $\phi_{sky_{eli_t}}$ ’ was upgraded to utilize the Stefan–Boltzmann law dynamically based on su

```

1 # Swinbank formula for Apparent Sky Temperature
2 T_air_K = sim_df["T2m"].iloc[Tstepi] + 273.15
3 T_sky_K = 0.0552 * (T_air_K ** 1.5)
4
5 sigma = 5.67e-8
6 epsilon_surf = 0.9
7 T_surf_K = Theta_old[ri] + 273.15
8
9 # Calculate actual non-linear sky radiation (W/m2) and update Matrix A
10 phi_sky_eli_t = sky_factor_elements[Eli] * epsilon_surf * sigma * ((
    T_surf_K ** 4) - (T_sky_K ** 4))
11 heat_radiative_elements_external[Eli] = 4.0 * epsilon_surf * sigma *
    (((T_surf_K + T_sky_K) / 2.0) ** 3)

```

6 Transient Heat Accumulation (Thermal Mass Skew)

6.1 Reasoning

The rigid RC-network node distribution (Class D) distributes thermal mass equally across wall layers (e.g., 25% per inner node). This fails to accurately mimic real-world thermal lag, causing instant cooling load spikes when solar radiation hits the exterior.

6.2 Code Implementation

In `Areal_heat_capacity_of_element`, a balanced, custom node weight array was assigned to skew capacitance to buffer against solar spikes.

```

1 # Balanced RC-network distribution for Class D
2 node_weights = [0.125, 0.25, 0.25, 0.25, 0.125]
3
4 for i in range(len(el_type)):
5     if el_type[i] == "OP" or el_type[i] == "ADJ":
6         for node in range(5):
7             kappa_pli_eli_[node, i] = list_kappa_el[i] * node_weights[
                node]

```

7 Coupling of Thermally Conditioned Zones

7.1 Reasoning

The standard standardizes multi-zone buildings without thermal coupling (assuming zero heat transfer between conditioned zones). For residential apartments, heat bleeding through adjacent walls to uncooled corridors or adjacent units is a significant physical mechanism.

7.2 Code Implementation

A dynamic heat bleed calculator was added to the internal heat gains loop.

```
1 Phi_tr_adj_zones = 0.0
2 for adj_zone in building_object.get("adjacent_zones", []):
3     T_adj_zone = 20.0 # Baseline or dynamic schedule
4     T_current_zone = Theta_int_old
5
6     for area, u_val in zip(adj_zone["area_facade_elements"], adj_zone["
    transmittance_U_elements"]):
7         heat_bleed = u_val * area * (T_adj_zone - T_current_zone)
8         Phi_tr_adj_zones += heat_bleed
9
10 Phi_int_z_t += Phi_tr_adj_zones
```

8 Subsequent changes in the heating/cooling load

Q_H (kWh)	Q_C (kWh)	Q_{DHW} (kWh)	Q_{Latent} (kWh)	Q_{total} (kWh)
2115.5	1075	1138.379	284.3281	4613.207
2486	888	1138.379	284.3281	4796.707
2491	864	1138.379	284.3281	4777.707
2513	818	1138.379	284.3281	4753.707
2597.402	723.6886	1138.379	284.3281	4743.797

Table 1: Annual thermal energy needs demonstrating the progressive decrease in the cooling load.

References

- [1] Ballarini, I., Costantino, A., Fabrizio, E., & Corrado, V. (2020). *A Methodology to Investigate the Deviations between Simple and Detailed Dynamic Methods for the Building Energy Performance Assessment*. *Energies*, 13(23), 6217. <https://doi.org/10.3390/en13236217>
- [2] Zakula, T., Badun, N., Ferdelji, N., & Ugrina, I. (2021). *Framework for the ISO 52016 standard accuracy prediction based on the in-depth sensitivity analysis*. *Applied Energy*, 298, 117089. <https://doi.org/10.1016/j.apenergy.2021.117089>
- [3] De Luca, G., Bianco Mauthe Degerfeld, F., Ballarini, I., & Corrado, V. (2022). *Improvements of simplified hourly models for the energy assessment of buildings: The application of EN ISO 52016 in Italy*. *Energy Reports*, 8, 7349-7359. <https://doi.org/10.1016/j.egyr.2022.05.120>